



OFPPT

RAPPORT DE PROJET

BOON - Gestion des Documents de Construction

Par : Amine Nabou

Promotion : DevOWFS 204

Établissement : OFPPT

Année : 2026

Rapport de Projet : BOON - Gestion des Documents de Construction

Table des Matières

1. Introduction
2. Contexte et Problématique
3. Objectifs du Projet
4. Fonctionnalités Principales
5. Architecture Technique
6. Conception et Modélisation
7. Réalisation
8. Démonstration
9. Difficultés Rencontrées et Solutions
10. Résultats Obtenus
11. Perspectives d'Amélioration
12. Compétences Développées
13. Conclusion

1. Introduction

BOON est une application web collaborative dédiée à la gestion des documents dans le secteur du BTP (Bâtiment et Travaux Publics). Elle permet aux propriétaires de chantiers, aux ouvriers et aux fournisseurs de centraliser, partager et gérer les documents clés (bons de commande, factures, devis) dans des espaces de travail virtuels appelés "rooms".

2. Contexte et Problématique

Dans le secteur du BTP, la gestion des documents est souvent effectuée manuellement ou via des outils peu adaptés, ce qui cause :

- **Perte de documents** : Gestion papier risquée.
- **Manque de traçabilité** : Impossible de suivre l'historique des modifications et des partages.
- **Communication compliquée** : Coordination difficile entre les différents acteurs (propriétaires, ouvriers, fournisseurs).
- **Erreurs humaines** : Saisie incorrecte, duplications, etc.

Problématique

Comment simplifier et centraliser la gestion des documents de construction pour améliorer la collaboration entre les acteurs d'un chantier ?

3. Objectifs du Projet

Les objectifs fixés pour ce projet sont :

1. **Centraliser les documents** : Stocker tous les documents dans un seul endroit sécurisé.
2. **Améliorer la collaboration** : Permettre aux acteurs de partager des documents rapidement.
3. **Générer des PDFs automatiquement** : Offrir des documents professionnels prêts à être partagés.
4. **Gestion des rôles** : Contrôler l'accès aux informations selon le rôle de l'utilisateur.
5. **Suivi analytique** : Donner un aperçu des dépenses et des documents.

4. Fonctionnalités Principales

BOON propose les fonctionnalités suivantes :

4.1 Gestion des Utilisateurs et Authentification

- Inscription avec numéro de téléphone et vérification par code.
- Connexion via identifiants.
- Gestion des tokens (accès et rafraîchissement).

4.2 Gestion des Rooms (Salles de Chantier)

- Création de rooms par le propriétaire.
- Code de room unique pour inviter d'autres utilisateurs.
- Demandes d'adhésion pour les ouvriers.
- Approbation/rejet des demandes par le propriétaire.
- Mise à jour du statut de la room (active/fermée).

4.3 Gestion des Documents

- Création de documents : bon de commande, facture, devis.
- Deux types de documents :
- **Personnels** : Visibles seulement par le fournisseur créateur.
- **Partagés** : Dans une room, visibles par les acteurs concernés.
- Ajout d'articles (nom, quantité, prix unitaire).
- Montant rapide si pas d'articles.
- Pièces jointes (photos, fichiers).
- Génération de PDF professionnel.

4.4 Lien Ouvrier - Fournisseur

- Les ouvriers peuvent lier des fournisseurs à une room.
- Les ouvriers voient seulement les documents des fournisseurs qu'ils ont liés.

4.5 Profil Fournisseur

- Gestion du profil professionnel (nom du magasin, téléphone, adresse, ICE, RC).

- Logo et note de pied de page pour les PDFs.

4.6 Tableau de Bord et Analytics

- Vue d'ensemble des montants totaux.
- Détails par type de document.
- Détails par catégorie.
- Top des fournisseurs.
- Historique des 7 derniers jours.

4.7 Partage des Documents

- Génération de liens temporaires signés valides 7 jours.
- Partage direct via WhatsApp.

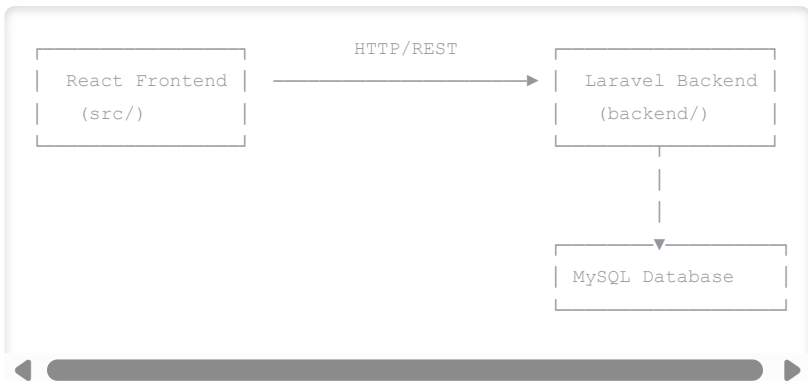
5. Architecture Technique

5.1 Stack Technique

BOON utilise une architecture **3 tiers** (frontend, backend, base de données) avec les technologies suivantes :

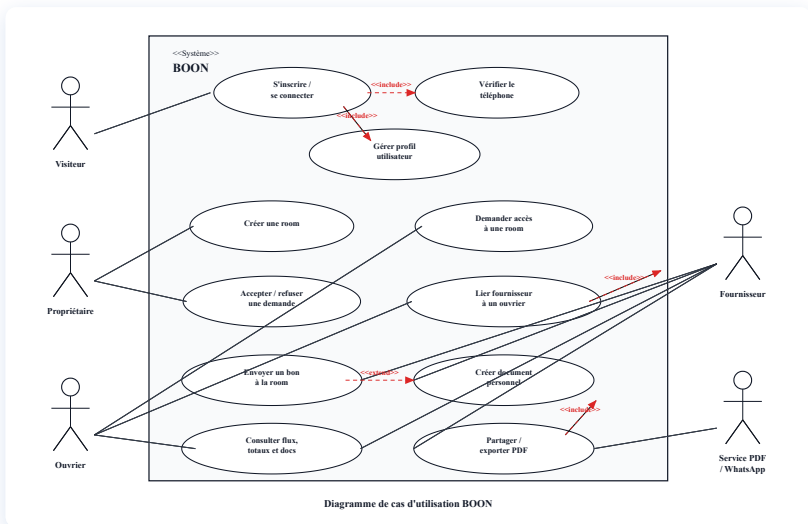
Couche	Technologies
Frontend	React 19, JavaScript, Tailwind CSS, Vite
Backend	Laravel 12, PHP 8.x
Base de données	MySQL
Autres outils	barryvdh/laravel-dompdf (génération PDF), Git (gestion de versions)

5.2 Diagramme d'Architecture



6. Conception et Modélisation

6.1 Diagramme de Cas d'Utilisation



Les principaux cas d'utilisation sont :

1. **Acteur : Propriétaire**
2. Créer une room
3. Voir les demandes d'adhésion

- 4. Approuver/Rejeter une demande
- 5. Gérer les membres de la room
- 6. Voir tous les documents de la room
- 7. Voir les analytics
- 8. **Acteur : Ouvrier**
 - 9. Rejoindre une room via le code
 - 10. Lier un fournisseur à une room
 - 11. Voir les documents des fournisseurs liés
- 12. **Acteur : Fournisseur**
 - 13. Créer/Modifier son profil
 - 14. Créer des documents personnels
 - 15. Créer des documents dans une room
 - 16. Télécharger le PDF d'un document
 - 17. Partager un document

6.2 Diagramme Entité-Relation (ER)

Voici les tables principales de la base de données (voir [UML/diagramme_classes.svg](#) pour le diagramme complet) :

Table : `users`

Colonne	Type	Contrainte	Description
<code>id</code>	UUID	PK	Identifiant unique
<code>phone</code>	VARCHAR	Unique	Numéro de téléphone
<code>email</code>	VARCHAR	Nullable	Adresse e-mail
<code>password_hash</code>	VARCHAR	-	Mot de passe haché

Colonne	Type	Contrainte	Description
<code>full_name</code>	VARCHAR	-	Nom complet
<code>default_role</code>	ENUM	-	Rôle : OWNER, WORKER, SUPPLIER
<code>phone_verified_at</code>	TIMESTAMP	Nullable	Date de vérification du téléphone
<code>status</code>	ENUM	-	Statut : ACTIVE, DISABLED
<code>created_at</code>	TIMESTAMP	-	Date de création
<code>updated_at</code>	TIMESTAMP	-	Date de modification

Table : `rooms`

Colonne	Type	Contrainte	Description
<code>id</code>	UUID	PK	Identifiant unique
<code>name</code>	VARCHAR	-	Nom de la room
<code>room_code</code>	VARCHAR	Unique	Code de la room
<code>status</code>	ENUM	-	Statut : ACTIVE, CLOSED
<code>owner_id</code>	UUID	FK → users.id	Propriétaire de la room
<code>last_activity_preview</code>	TEXT	Nullable	Aperçu de la dernière

Colonne	Type	Contrainte	Description
			activité
<code>last_activity_at</code>	TIMESTAMP	Nullable	Date de la dernière activité
<code>created_at</code>	TIMESTAMP	-	Date de création
<code>updated_at</code>	TIMESTAMP	-	Date de modification

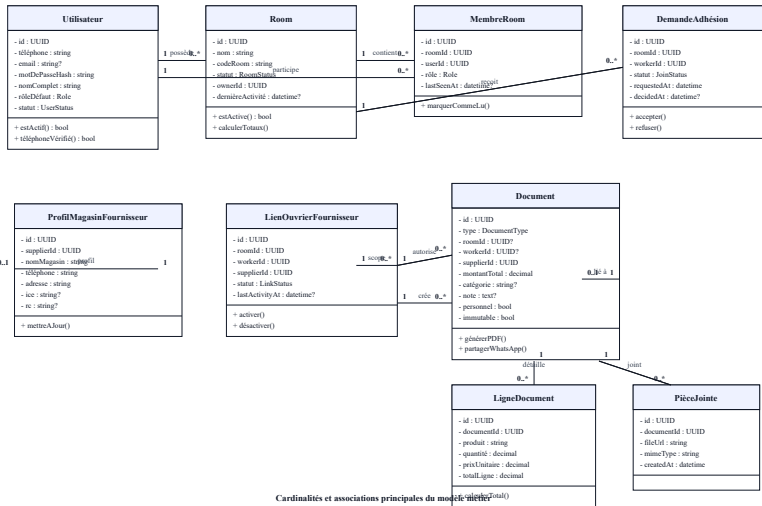
Table : documents

Colonne	Type	Contrainte	Descript
<code>id</code>	UUID	PK	Identifica unique
<code>type</code>	ENUM	-	Type : RECEIPT INVOICI QUOTE
<code>room_id</code>	UUID	Nullable, FK → rooms.id	Room associée
<code>worker_id</code>	UUID	Nullable, FK → users.id	Ouvrier associé
<code>supplier_id</code>	UUID	FK → users.id	Fourniss créateur
<code>created_by_supplier_id</code>	UUID	FK → users.id	Fourniss original

Colonne	Type	Contrainte	Descript
<code>store_profile_id</code>	UUID	FK → supplier_store_profiles.id	Profil fourniss
<code>quick_amount</code>	DECIMAL	Nullable	Montan rapide
<code>category</code>	VARCHAR	Nullable	Catégor
<code>note</code>	TEXT	Nullable	Note
<code>currency</code>	VARCHAR	-	Devise (défaut: MAD)
<code>grand_total</code>	DECIMAL	-	Montan total
<code>is_personal</code>	BOOLEAN	-	Docume personn
<code>immutable</code>	BOOLEAN	-	Non modifia ?
<code>created_at</code>	TIMESTAMP	-	Date de création

6.2 Diagramme Entité-Relation et Classes

Diagramme de classes BOON



Cardinalités et associations principales du modèle métier

6.3 Autres Tables Clés

- **api_tokens** : Gestion des tokens d'accès et de rafraîchissement
- **room_members** : Lien entre users et rooms avec rôle
- **room_join_requests** : Demandes d'adhésion aux rooms
- **worker_supplier_links** : Lien entre ouvriers et fournisseurs dans une room
- **supplier_store_profiles** : Profils professionnels des fournisseurs
- **document_items** : Articles des documents
- **attachments** : Pièces jointes des documents
- **phone_verification_codes** : Codes de vérification du téléphone

7. Réalisation

7.1 Backend (Laravel)

Le backend est structuré en :

- **Controllers** : Traitement des requêtes API.

- **BoonAuthController** : Authentification (inscription, connexion, vérification, rafraîchissement)
- **BoonRoomController** : Gestion des rooms
- **BoonDocumentController** : Gestion des documents et PDFs
- **BoonSupplierController** : Gestion des fournisseurs et liens
- **BoonAnalyticsController** : Analytics
- **Models** : Eloquent (seulement User utilisé, DB Query Builder pour le reste)
- **Middleware** : **BoonAuthenticate** (vérifie le token) et **BoonRequireRole** (vérifie le rôle)
- **Support** : **BoonApiSupport** (trait avec toute la logique métier)
- **Views** : Blade pour le template PDF (`pdf.document`)

7.2 Frontend (React)

Le frontend est organisé en :

- **Components** :
 - **AuthGate** : Gestion de l'authentification
 - **Dashboard** : Page d'accueil
 - **RoomLive** : Vue d'une room
 - **BoonCenter** : Documents personnels du fournisseur
 - **Profile** : Profil utilisateur et fournisseur
 - **Reports** : Analytics
 - **BottomNav** : Navigation du bas
- **API** : **api.js** (types en commentaires si besoin) et **api-rest.js** (requêtes HTTP)
- **Styles** : Tailwind CSS avec thème clair/sombre et support RTL (Arabic)

8. Démonstration

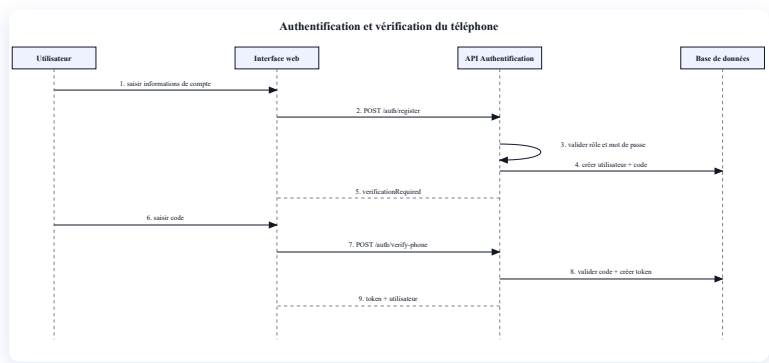
8.1 Captures d'écran de l'application

(Ajoutez ici les captures d'écran des différentes pages de l'application)

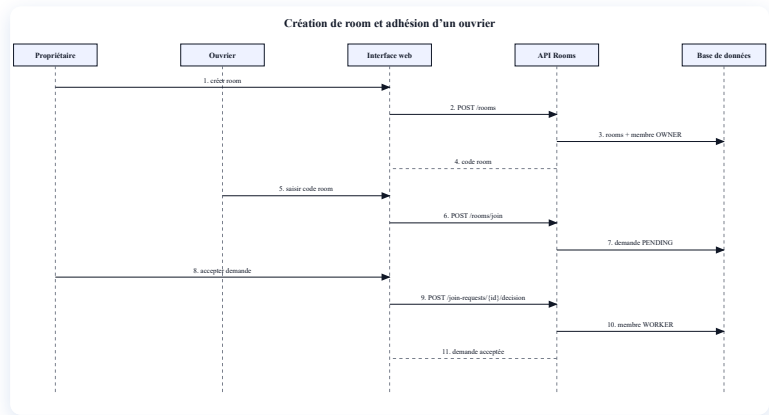
Scénario Principal

Diagrammes de Séquence

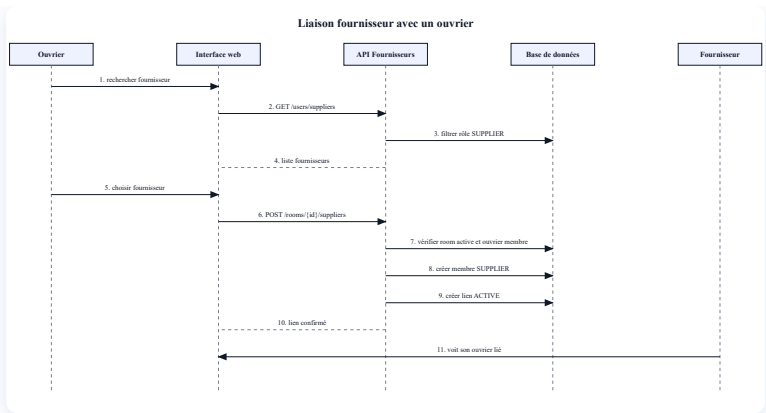
1. Authentification



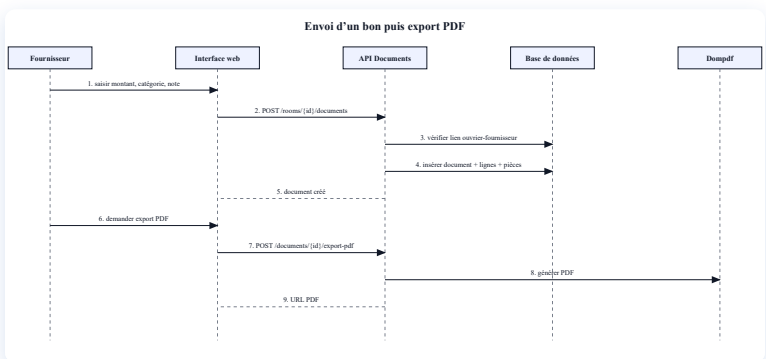
2. Création et Rejoindre une Room



3. Liaison Fournisseur



4. Création et Génération de Bon/PDF



Voici un scénario d'utilisation typique de BOON :

1. Inscription et Vérification

2. Amine (propriétaire) s'inscrit avec son numéro de téléphone.
3. Il reçoit un code de vérification et valide son compte.

4. Création d'une Room

5. Amine crée une room appelée "Chantier Villa Agadir".
6. Il obtient un code unique : "VIL-ABC123".

7. Adhésion de l'Ouvrier

8. Youssef (ouvrier) s'inscrit et se connecte.

9. Il rejoint la room en utilisant le code.
10. Amine reçoit une notification et approuve la demande.
11. **Liaison du Fournisseur**
12. Youssef recherche et lie le fournisseur "Matériaux Souss".
13. "Matériaux Souss" est automatiquement ajouté à la room.
14. **Création d'un Devis**
15. "Matériaux Souss" crée un devis pour des ciments et briques.
16. Il sélectionne Youssef comme ouvrier associé.
17. **Téléchargement et Partage**
18. Le devis est disponible pour Amine et Youssef.
19. "Matériaux Souss" télécharge le PDF et partage le lien via WhatsApp.

9. Difficultés Rencontrées et Solutions

9.1 Authentification Sécurisée

- **Problème** : Gérer des sessions sécurisées sans utiliser Laravel Sanctum ou Passport.
- **Solution** : Mise en place d'un système personnalisé avec tokens d'accès (15 min) et tokens de rafraîchissement (30 jours), stockés dans la table `api_tokens` .

9.2 Génération PDF Responsive

- **Problème** : Générer des PDFs professionnels avec les données du fournisseur et les articles.
- **Solution** : Utilisation de la bibliothèque `barryvdh/laravel-dompdf` et un template Blade (`resources/views/pdf/document.blade.php`).

9.3 Gestion des Droits d'Accès par Rôle

- **Problème** : Contrôler qui peut voir/faire quoi dans l'application.

- **Solution** : Middleware `BoonRequireRole` qui vérifie le rôle de l'utilisateur avant chaque requête protégée, et logique métier dans `BoonApiSupport` pour filtrer les documents visibles.

9.4 Traçabilité des Activités

- **Problème** : Savoir qui a fait quoi et quand dans une room.
- **Solution** : Mise à jour automatique du champ `last_activity_at` et `last_activity_preview` de la room à chaque création de document.

10. Résultats Obtenus

À l'issue du développement :

1. **MVP Fonctionnel** : Toutes les fonctionnalités principales sont implémentées et utilisables.
2. **Code Organisé** : Architecture claire, séparation frontend/backend.
3. **Sécurité** : Authentification sécurisée, hachage des mots de passe (Hash facade Laravel), vérification des droits d'accès.
4. **Expérience Utilisateur** : Interface intuitive, responsive, support RTL.
5. **Tests Réalisés** : Testé avec les trois rôles (propriétaire, ouvrier, fournisseur).

11. Perspectives d'Amélioration

Pour continuer à améliorer BOON, plusieurs pistes sont envisageables :

1. **Déploiement Cloud** : Héberger l'application sur un service cloud (Render, AWS, etc.) pour qu'elle soit accessible en ligne.
2. **Notifications Push** : Avertir les utilisateurs des nouveaux documents ou demandes.
3. **Mode Hors Ligne** : Permettre de créer des documents sans connexion, synchronisation ultérieure.
4. **Historique des Modifications** : Garder une trace des modifications des documents.

5. **Application Mobile** : Créer une application mobile iOS/Android pour une meilleure accessibilité sur les chantiers.
6. **Multidevise** : Support de plusieurs devises (EUR, USD, etc.).
7. **Intégrations** : Intégrer avec d'autres outils comptables (QuickBooks, Sage, etc.).

12. Compétences Développées

Ce projet a permis de développer les compétences suivantes :

- Développement **Full Stack** : React + Laravel.
- **Conception UML** : Diagrammes de classes, cas d'utilisation, séquence.
- **Gestion de Base de Données** : Modélisation, migrations, requêtes SQL.
- **Sécurité** : Authentification, gestion des droits, hachage.
- **Gestion de Versions** : Git, GitHub.
- **UX/UI** : Design d'interfaces responsives avec Tailwind CSS.
- **Résolution de Problèmes** : Analyse des défis et trouver des solutions appropriées.

13. Conclusion

BOON est une réponse concrète aux problèmes de gestion de documents dans le secteur du BTP. Grâce à son architecture moderne, ses fonctionnalités collaboratives et son interface intuitive, BOON permet aux acteurs d'un chantier de travailler plus efficacement et avec une meilleure traçabilité. Le MVP est complet et fonctionnel, et de nombreuses améliorations possibles permettent d'envisager une évolutivité à long terme.

Auteur : Amine Nabou

Promotion : DevOWFS 204

Établissement : OFPPT

Année : 2026

